Report 3: Loggy

Rodrigo Perestrelo, Lennard Last
ID2201 Distributed Systems, Basic Course

September 23, 2026

1 Introduction

The objective of this assignment is to design and evaluate `Loggy`, a centralized logging service that receives event streams from concurrent worker processes. Because message delivery across separate Erlang actor channels is asynchronous, messages frequently arrive at the logger out of causal order. We address this challenge by introducing an abstract logical `time` module, augmenting workers with Lamport timestamps, and implementing a holdback queue within the logger to buffer events until they can be safely printed in causal sequence.

2 The Time Module and Architectural Modifications

To maintain strict modularity, logical clock semantics are encapsulated entirely inside `time.erl`. In our Lamport implementation, time is modeled as an integer counter starting at 0 (`zero/0`). The module provides arithmetic for workers to advance (`inc/2`), merge causally upon message receipt (`merge/2`), and compare timestamps (`leq/2`). For the logger, it manages a list of worker-timestamp pairs (`clock/1`), updates them upon event arrival (`update/3`), and checks whether all nodes have advanced past a given timestamp (`safe/2`). Neither the workers nor the logger perform direct clock arithmetic or inspect internal representations.

2.1 Modifications to the Worker Process

In the naive baseline, workers possessed no awareness of time, tagging all outbound log entries with a placeholder atom (`na`). To enable causal tracking, we enhanced the worker process loop in three key aspects:

1. **State Augmentation:** The worker loop was modified to maintain an internal logical clock initialized via `time:zero()`.

2. **Causal Synchronization on Message Receipt:** When receiving a peer message `{msg, MsgTime, Msg}`, the worker cannot simply trust its local clock. It updates its state by taking the causal merge of its internal time and the incoming timestamp, followed by an increment (`time:inc(Name, time:merge(Time, MsgTime))`). This ensures the worker's clock immediately advances past the sender's event horizon.

3. **Timestamp Piggybacking on Transmission:** When sending a peer message, the worker advances its local clock (`time:inc/2`) and tags both the network message dispatched to the peer and the event log dispatched to the logger with this updated timestamp.

2.2 Modifications to the Logger

The initial logger was entirely passive and unbuffered: upon receiving any log event, it executed an immediate `io:format/2` to stdout. We re-engineered the logger into an active ordering coordinator:

1. **Global Worker Progress Tracking:** The logger was given an initial list of active worker identifiers (`Nodes`). It initializes an internal tracking structure (`Clock`) that records the latest logical timestamp received from every node in the system.

2. **Holdback Queue Buffering:** Instead of printing immediately, incoming log events are placed into a holdback queue sorted in ascending order of logical timestamp using `time:leq/2`.

3. **Conditional Safe Flush:** Upon receiving any new entry and updating the sender's clock entry, the logger inspects the front of the queue. An event with timestamp T is released and printed if and only if `time:safe(T, Clock)` confirms that every monitored worker has reported a timestamp $\geq T$.

2.3 System-Wide Architectural Impact

These additions fundamentally alter the system dynamics:

- **Decoupling Delivery from Ingestion:** The logger's display order is completely decoupled from the non-deterministic arrival order over Erlang's asynchronous channels. Network delays and processing jitter no longer corrupt the observed event chronology.

- **Strict Modularity via Abstract API:** Neither the worker nor the logger contains integer arithmetic, list manipulation, or pattern matching tied to Lamport representation. The worker only knows how to increment and merge timestamps; the logger only knows how to register progress and query event safety. This allows replacing scalar Lamport clocks with vector clocks without changing a single line of worker or logger application logic.

3 Analysis of the Initial Implementation and Causal Order

3.1 Empirical Detection of Out-of-Order Events and Jitter Impact

In the naive logger, workers dispatched log messages immediately upon receiving or sending a peer message, while an artificial delay (`jitter`) was injected between peer delivery and log dispatch. Because Erlang only guarantees FIFO delivery pairwise between two specific processes, the arrival order at the logger across multiple workers was non-deterministic.

When executing the unbuffered logger with `test:run(50, 200)`, out-of-order execution was detected in two ways:

```
log:  2 john {received,{hello,91}}
log:  2 george {received,{hello,27}}
log:  4 george {received,{hello,51}}
log:  2 john {sending,{hello,51}}
log:  0 ringo {sending,{hello,27}}
log:  6 ringo {received,{hello,61}}
log:  0 paul {sending,{hello,91}}
```

1. **Causal Inversion (Receive before Send):** At line 1, `john` logged the receipt of message `{hello, 91}` at logical time 2. However, the matching transmission event from `paul` at

time 0 did not print until line 7. A cause cannot occur after its effect; the receipt log arrived at the logger before the sender's log due to network jitter.

2. **Monotonicity Inversion:** Between lines 3, 4, and 5, the logged timestamps printed to the screen were 4, 2, and 0. Time flowed backwards in the output stream.

Can Jitter Eliminate Out-of-Order Entries?
Experimenting with the jitter parameter demonstrates that increasing jitter widens the latency variance between peer delivery and logging, significantly increasing the frequency of out-of-order arrivals. Conversely, reducing jitter decreases this probability. However, setting jitter to 0 **cannot eliminate out-of-order prints entirely**. Because the BEAM virtual machine preemptively schedules Erlang processes across multiple operating system threads, non-deterministic execution scheduling can still cause a receiving worker's log to reach the logger before the sending worker is scheduled to dispatch its own log message.

3.2 Causal Guarantees vs. Physical Reality

- **What is always true?** Lamport's clock condition guarantees that if event A causally precedes event B ($A \to B$), then $L(A) < L(B)$. Consequently, every send event strictly precedes its matching receive event in logical time ($L(\text{send}) < L(\text{receive})$). Furthermore, event timestamps local to any single worker process are strictly monotonic.

- **What is sometimes true?** If $L(A) < L(B)$, it is only **sometimes true** that A physically preceded B or causally influenced B. If A and B occurred on separate workers without message exchange, they are concurrent ($A \parallel B$). Lamport timestamps linearize concurrent events into an arbitrary total order that respects causality, but does not reflect real-time physical concurrency.

- **How do you play it safe?** Since an individual worker's events are strictly monotonic, once the logger observes a timestamp T_k from worker k, that worker will never emit an event with timestamp $< T_k$. Therefore, an event at timestamp T in the holdback queue is safe to release only when **all** monitored workers have reported a timestamp $\geq T$.

- **What does the final log tell us, and did events happen in that order?** The final log presents a valid **total ordering consistent with causality** (a topological sort of the distributed system's causal graph). Causally dependent events are guaranteed to have occurred in the order logged. However, for concurrent events, the log order is an artifact of Lamport tie-breaking and does not imply that events occurred in that exact sequence in physical wall-clock time.

4 Implementation: Lamport Clocks and Holdback Queue

4.1 Holdback Queue Mechanics

The logger maintains the worker clock map, a timestamp-sorted holdback queue, and a high-water mark tracking maximum queue size. Incoming messages are inserted into the queue in ascending order using `time:leq/2`. The queue is then processed sequentially: safe items are flushed and logged, while processing halts at the first unsafe entry.

```
loop(Clock, Queue, MaxLen) ->
    receive
        {log, From, Time, Msg} ->
            NewClock = time:update(From, Time, Clock),
            NewQueue = insert_sorted({log, From, Time, Msg}, Queue),
            UpdatedMax = erlang:max(MaxLen, length(NewQueue)),
```

```
            RemainingQueue = flush_safe(NewClock, NewQueue),
            loop(NewClock, RemainingQueue, UpdatedMax);
        stop ->
            io:format("Logger stopped. Remaining unsent in queue: ~p~n", [
    length(Queue)]),
            ok
    end.
```

4.2 Experimental Evaluation and Queue Dynamics

We evaluated queue retention across three representative test configurations:

Configuration	Sleep (ms)	Jitter (ms)	Max Queue Entries	Remaining at Stop
Low Jitter	100	10	4	8
Moderate Jitter	50	500	15	12
High Jitter	10	1000	28	12

Table 1: Holdback queue performance under varying latency conditions.

Analysis of Results:

- **Low Jitter (`Sleep = 100, Jitter = 10`):** Workers report actions to the logger almost immediately after peer communication. Clocks across all nodes advance evenly, enabling the logger to release messages rapidly. The queue remains shallow, peaking at 4 entries.
- **High Jitter (`Sleep = 10, Jitter = 1000`):** With high jitter and rapid message transmission, one worker's log entry may be substantially delayed while other nodes continue exchanging messages and advancing their Lamport clocks. Because the logger cannot flush an entry until the slowest node catches up, events from faster nodes accumulate, causing the holdback queue to swell to 28 entries.
- **Remaining Entries at Termination:** In all executions, a small residual queue remains unprinted when `stop` is received. Because the simulation halts abruptly, active workers cease transmitting, leaving the logger unable to advance the clocks of lagging nodes to confirm safety for the final buffered entries.

5 Optional Task: Vector Clocks (Bonus)

The optional task of the assignment was to implement a vector clock.

5.1 Comparison: Lamport Clocks vs. Vector Clocks

A scalar Lamport timestamp collapses causal history into a single integer, which is sufficient to guarantee $A \to B \implies L(A) < L(B)$, but not the converse: two unrelated, concurrent events may receive any relative timestamp. Consequently, `safe/2` for Lamport clocks must conservatively wait for *every* monitored worker to advance past T, since a scalar value carries no information about which nodes an event actually depends on. A vector clock replaces the single counter with one counter per process, encoding precisely which nodes causally contributed to a given event and how many of their events did so. This allows `leq/2` to determine not just an order between two timestamps, but whether two events are causally related at all, or concurrent. The practical consequence for the logger is that `safe/2` only needs confirmation from the nodes named in a given event's vector, rather than from the whole system.

5.2 Implementation Details

Vector time is represented as a list of `{Name, Counter}` pairs rather than a fixed-size tuple, keeping the representation independent of the number of nodes in the system. `zero/0` returns `[]`, meaning that no events from any process have yet been observed. `inc/2` increments a process's own entry, adding it if absent. `merge/2` takes the pairwise maximum of matching entries, carrying through entries present in only one of the two vectors unchanged. `leq/2` performs a componentwise comparison, treating any entry absent from the other vector as implicitly zero. `clock/1` similarly starts as `[]`: unlike the Lamport clock, it does not need to be seeded with known node names, since `update/3` adds entries the first time each node is heard from. Lastly, `safe/2` reduces to exactly the same componentwise condition as `leq/2` so it is implemented as a direct call to `leq/2`.

5.3 Experimental Results and Benefits

Measured under the same conditions as the Low Jitter configuration in Table 1, the average holdback queue length dropped from around 7 entries with Lamport clocks to less than 1 with vector clocks. This is a direct result of the narrower safety condition: Lamport's `safe/2` requires all nodes to catch up before any event can be released, so the queue is bottlenecked by whichever node is currently slowest, regardless of relevance. Vector clocks only require the specific nodes named in an event's own vector to have caught up, so most events become safe almost as soon as their directly relevant sender and receiver have reported in. The tradeoff is that each timestamp grows with the number of active nodes rather than remaining a single integer, and comparisons are componentwise rather than a single integer comparison.

6 Conclusions

This laboratory confirmed that asynchronous message passing across multiple actor processes cannot ensure causal consistency without explicit ordering mechanisms. While Erlang provides FIFO guarantees point-to-point, multi-process dependencies require logical timestamps. Implementing Lamport time across decoupled `worker`, `time`, and `mylogger` modules resolved causal inversions, ensuring that all log entries output in a valid total order consistent with causal precedence.